

FIT5217 - Natural Language Processing

Assignment 2: Recipe Generation and Menu Design

Monash University

Total marks:	100
Weight:	25% of final grade
Released:	Wednesday, Week 7, 22 April 2026
Due:	Tuesday, Week 12, 26 May 2026 — 11:55 PM
Interview:	During Week 12 Lab
Submission:	via Moodle in one ZIP file
Late penalty:	5% per day (max 7 days)
Authorship:	Individual

Overview

In this assignment, you will build and evaluate a family of models for **recipe generation** — the task of producing step-by-step cooking instructions given a list of ingredients. For example, given a simple set of ingredients such as brown sugar, milk, and butter, the model should be able to generate a plausible recipe, such as instructions for preparing brown sugar fudge:

```
Input (Ingredients):  
["1 c. brown sugar", "1/2 c. milk", "2 tbsp. butter"]  
  
Output (Recipe):  
["Combine ingredients in saucepan.", "Boil 5 minutes,  
  stirring constantly.", "Cool and serve."]
```

The journey spans three milestones that mirror the modern NLP stack:

- **Task 1 (24 marks):** Train an encoder-decoder RNN from scratch in PyTorch, first without attention, then with Bahdanau attention.
- **Task 2 (27 marks):** Fine-tune two pre-trained language models (T5 and GPT-2) using Low-Rank Adaptation (LoRA) [7], and compare against each other and your Task 1 baselines.
- **Task 3 (25 marks):** Build an intelligent **Menu Designer** that retrieves relevant recipes from a large corpus using Retrieval-Augmented Generation (RAG) [8], generates multi-dish menus satisfying dietary and cultural constraints via a free LLM API, and evaluates its own outputs using an LLM-as-judge framework.
- **Interview (24 marks):** A 10–15 minute Week-12 interview covering implementation review for Tasks 1–3 (19 marks) and a live demo of the Menu Designer (5 marks).

Dataset

All three tasks use the **Cooking Dataset** [2], which is included in the assignment's data folder. The dataset statistics are shown below for reference.

Split	File	Rows(w/ header)	Purpose
Training	<code>train.csv</code>	162,900	Train models
Development	<code>dev.csv</code>	1,066	Tune hyperparameters; monitor training
Test	<code>test.csv</code>	1,082	Final evaluation only

Each row has three columns:

Column	Type	Description
Title	string	Recipe name (not used for Tasks 1–2)
Ingredients	JSON list	Ingredients with quantities, e.g. ["1 c. brown sugar", "1/2 c. milk"]
Recipe	JSON list	Ordered cooking steps, e.g. ["Combine ingredients.", "Boil 5 min."]

Important:

- The test split must *not* be used for any training, preprocessing, or vocabulary decisions. All preprocessing choices must be made using the training split only.
- In task 3, you can build a retrieval index over the full dataset (train + dev + test) since the RAG pipeline is not trained on the data.

Compute Resources

All experiments in this assignment are run on **Google Colab**. You are allocated a Google Colab Pro subscription with **100 compute units per calendar month**. The table below summarizes the available GPU tiers and their unit costs; plan your usage carefully as units are shared across all tasks and any exploratory runs. A step-by-step Colab guide is available [here](#):

GPU	Units / hour	VRAM (GB)	Suggested use
T4	1.41	16.0	Task 1, Task 3
L4	1.71	22.5	Task 1, Task 3
A100	7.52	80.0	Task 2 long runs (enable <i>High RAM</i>)
G4	8.71	95.6	Task 2 long runs (enable <i>High RAM</i>)

Tips for conserving units:

- Select the cheapest GPU tier that fits your model in memory.
- Debug and tune hyperparameters on a small data subset before running full training.
- **Mount your Google Drive first** (`drive.mount('/content/drive')`) before writing any files.
- **Save checkpoints and outputs to Google Drive**, not to the Colab temporary environment — files in `/content/` are lost when the session ends.
- Save model checkpoints frequently (`torch.save`) so a session timeout does not force a full rerun.
- Pre-build and persist any indexes or embeddings to disk so they can be reloaded without recomputation.

Task 1 — RNN Recipe Generation from Scratch [24 marks]

Goal: Implement an RNN-based sequence to sequence architecture using PyTorch, trained from scratch on the Cooking Dataset to generate recipes from ingredients.

T1.1 Data Preprocessing (2 marks)

Implement a text preprocessing pipeline:

- Tokenize ingredient and recipe text.
- Build a vocabulary from the **training split only** with a frequency threshold of your choice; map tokens below the threshold to <UNK>.
- Add four special tokens: <PAD>, <UNK>, <SOS> (start-of-sequence), <EOS> (end-of-sequence).
- Convert ingredient and recipe sequences to integer ID sequences.

In your notebook, report your chosen frequency threshold and justification, the resulting vocabulary size, and the out-of-vocabulary (OOV) rate on the dev split. Include a **sanity-check cell** that asserts all four special tokens (<PAD>, <UNK>, <SOS>, <EOS>) exist in both `word2idx` and `idx2word`, and add a short Markdown cell (~100 words) describing your preprocessing pipeline.

Evaluation (2 marks — assessed via notebook)

- **1 mark** — Correct preprocessing pipeline: special tokens present in both mappings (verified by sanity-check cell), frequency filtering applied. Markdown cell describes the pipeline.
- **1 mark** — Vocabulary size, OOV rate, and frequency threshold reported and justified.

T1.2 Baseline RNN Encoder-Decoder without Attention (5 marks)

Implement a basic encoder-decoder RNN model with:

- **Encoder:** an embedding layer followed by a single- or multi-layer RNN/LSTM/-GRU that processes the ingredient tokens and produces a final hidden state.
- **Decoder:** a step-by-step token generator initialised from the encoder's final hidden state, using **teacher forcing** during training.
- **Training loop:** cross-entropy loss over the target recipe tokens (excluding <PAD> positions), gradient clipping, and an early stopping strategy of your choice.

Evaluate on the **test split** and report BLEU-4, METEOR, and BERTScore (see Section “Evaluation Metrics”, 7).

Evaluation (5 marks — assessed via notebook)

- **2 marks** — Training loop: loss curve retained in notebook, early stopping applied and explained.
- **3 marks** — BLEU-4, METEOR, and BERTScore reported on the test split with a short discussion (~100 words) interpreting the scores.

T1.3 RNN Encoder-Decoder with Bahdanau Attention (5 marks)

Extend your baseline model by adding **Bahdanau attention** [1].

You must implement Bahdanau attention yourself — do **not** use off-the-shelf implementations. Allowed libraries: `torch.nn`, `torch.nn.functional`. Disallowed: `torch.nn.MultiheadAttention`, HuggingFace attention modules, or any pre-built AdditiveAttention implementation.

Implementation requirement: Include a Markdown cell (~200 words) explaining, in your own words, the score function, softmax normalisation, and context vector computation.

Evaluate on the test split using the same three metrics. In addition, include **one visualization and discussion** of the attention weights (e.g. a heatmap for a sample recipe, which ingredient tokens the decoder attends to most).

Evaluation (5 marks — assessed via notebook)

- **2 marks** — BLEU-4, METEOR, and BERTScore reported on the test split with a short discussion (~100 words) interpreting the scores.
- **3 marks** — Attention visualization and discussion of what the attention mechanism learns.

T1.4 Analysis and Reflection

(12 marks)

Hint:

- You don't need to focus on optimizing for the best possible metrics in Task 1; the goal is to build a working model, understand its behavior, and compare the relative behaviors of the implementation.
- The L4 and T4 GPUs are sufficient for Task 1; the A100 is not required.
- It is recommended to spend 30 – 50 **units** on Task 1 to preserve budget for Task 2. A single run of each model (Baseline and Attention) with a moderate number of epochs (e.g. ≤ 25) is sufficient for evaluation and analysis.

In your notebook (Markdown cells) and final report, address the following:

- (1) **Hyperparameter choices (4 marks):** Report the values you used for embedding dimension, hidden dimension, number of layers, learning rate, batch size, and teacher forcing ratio. Justify each choice with experimental evidence (e.g. a small ablation).
- (2) **Baseline vs. Attention comparison (5 marks):** Compare the two models quantitatively (metrics table) and qualitatively (e.g. sample generated recipes). Where does attention help? Are there cases where both models fail similarly?
- (3) **Training Dynamics (3 marks):** Report the training dynamics for both models: how many epochs were needed, when early stopping triggered, and whether loss continued to decrease or plateaued. Did the dev loss correlate well with test performance? Did you observe overfitting? Support your discussion with the loss curves from your notebook.

Report structure: A template with the required sections and table formats is provided in `report_template.md` (see submission materials).

Evaluation (12 marks — assessed via notebook and report)

- **4 marks** — Hyperparameter justification supported by evidence or reasoning.
- **5 marks** — Substantive quantitative and qualitative comparison of Baseline vs. Attention.
- **3 marks** — Training dynamics discussion grounded in loss curves.

Task 2 — Pre-trained Model Fine-tuning with LoRA [27 marks]

Goal: Fine-tune two pre-trained language models using **Low-Rank Adaptation (LoRA)** from the `peft`¹ library. Compare their performance against each other and against your Task 1 models.

All fine-tuning uses the training split; dev for hyperparameter tuning; test for final reported results. Use the same three evaluation metrics as Task 1.

T2.1 T5 Fine-tuning with LoRA (6 marks)

T5 (Text-to-Text Transfer Transformer) [3] adopts a unified text-to-text paradigm, where each task (e.g. classification, translation, summarisation, or question answering) is framed as a mapping from an input text string to an output text string. This design eliminates the need for task-specific architectures or output heads, enabling a single model to handle diverse NLP tasks through consistent formatting. In this task, you will fine-tune `google-t5/t5-small`² (or `google-t5/t5-base`³ if compute allows) from Hugging Face, including:

- Format the input and output as a plain string. Use the pre-trained T5 tokenizer to encode inputs and decode outputs.
- Apply LoRA for parameter-efficient fine-tuning. Run and compare **at least three** LoRA configurations, varying `lora_r`, `lora_alpha`, or target modules (at least one hyperparameter must differ across runs).
- Make predictions on the test split and evaluate using BLEU-4, METEOR, and BERTScore.

Evaluation (6 marks — assessed via notebook)

- **3 marks** — BLEU-4, METEOR, BERTScore reported on test split with a short discussion (~100 words) interpreting the scores.
- **3 marks** — Comparison of at least three LoRA configurations with discussion of their effect on performance.

T2.2 GPT-2 Fine-tuning with LoRA (6 marks)

In this task, you will fine-tune `openai-community/gpt2`⁴ [4] from Hugging Face:

¹<https://github.com/huggingface/peft>

²<https://huggingface.co/google-t5/t5-small>

³<https://huggingface.co/google-t5/t5-base>

⁴<https://huggingface.co/openai-community/gpt2>

- GPT-2 is a decoder-only transformer, so both the conditioning context and the target text must be concatenated into a single token sequence. Your first task is to frame the input and output as a single sequence with a clear delimiter between ingredients and recipe steps to construct the training data.
- Apply LoRA for parameter-efficient fine-tuning. For GPT-2, **one** LoRA configuration is sufficient — the multi-variant exploration is only required for T5 in T2.1.
- Make predictions on the test split and evaluate the generation with different decoding strategies (e.g. greedy, beam search) using BLEU-4, METEOR, and BERTScore.

Evaluation (6 marks — assessed via notebook)

- **3 marks** — BLEU-4, METEOR, BERTScore reported on test split with a short discussion (~100 words) interpreting the scores.
- **3 marks** — Comparison of different decoding strategies and discussion of their impact on generation quality.

T2.3 Analysis and Reflection

(15 marks)

Write a substantive analysis in your report covering the following three areas.

(a) Cross-model comparison (6 marks)

Produce a comparison table for all trained models: T1.2 — **Baseline** RNN, T1.3 — **Attention** RNN, T2.1 — **T5-small** + LoRA, T2.1 — **T5-base** + LoRA (*if applicable*), T2.2 — **GPT-2** + LoRA. Report BLEU-4, METEOR, and BERTScore for each, using the following template. You only need to report the score of the best model you obtained under each task.

Model	BLEU-4	METEOR	BERTScore	Rank
T1.2 - Baseline RNN				
T1.3 - Attention RNN				
T2.1 - T5-small + LoRA				
T2.1 - T5-base + LoRA (<i>optional</i>)				
T2.2 - GPT-2 + LoRA				

Your analysis should discuss the following points:

- *Pre-trained vs. from-scratch models*: why pre-trained models (T2) outperform (or underperform) the baseline RNN trained from scratch (T1) on this task;
- *Architectural differences*: compare the encoder–decoder architecture of T5 with the decoder-only design of GPT-2. How do these differences help explain any performance gap observed in your results?;
- *Effect of model size (if applicable)*: If you experimented with **T5-base**, discuss how increasing model size influenced both performance and computational cost (e.g., training time, memory usage).

(b) LoRA configuration analysis (6 marks)

Explain the role of key LoRA hyperparameters, including the rank (`lora_r`), scaling factor (`lora_alpha`), and dropout. Describe how each of these settings influences the number of trainable parameters and the model's capacity.

Specify the target modules to which LoRA was applied for each model: T5 (e.g. `q`, `v`) and for GPT-2 (e.g. `c_attn`). Justify your choice of target layers, explaining why these were selected instead of alternatives (e.g. feed-forward layers). Finally, compare the fraction of trainable parameters introduced by your LoRA configuration across both models, and discuss any notable differences.

(c) Training dynamics (3 marks)

Describe the training dynamics for each model, including details such as the number of epochs required, when (if at all) early stopping was triggered, and whether the training and validation loss showed consistent improvement or plateaued over time. Analyse whether the development (validation) loss served as a reliable indicator of test performance. Comment on any signs of overfitting, **grounding your discussion in the training/validation loss plots included in your notebook.**

Evaluation (15 marks — assessed via report)

- **6 marks** — Cross-model comparison table with substantive architectural discussion (pre-trained vs. from-scratch, T5 vs. GPT-2 architecture, effect of model size if applicable).
- **6 marks** — LoRA analysis: hyperparameter roles, parameter counts, module selection justification, efficiency comparison.
- **3 marks** — Training dynamics discussion grounded in training/validation loss plots.

Hint:

- You **don't** need to focus on optimizing for the best possible metrics in Task 2; the goal is to build working fine-tuned models and understand their behavior in comparison to each other and to Task 1.
- It is recommended to spend **less than 60 units on Task 2**. The fine-tuning runs can be relatively short (e.g. 5–10 epochs).
- A report template is provided in `report_template.md` to guide your T2.3 analysis.

Task 3 — Menu Designer: RAG and LLM-as-Judge [25 marks]

Goal: Build a **Menu Designer** that accepts a natural language request with dietary or cultural constraints, retrieves relevant recipes from the Cooking Dataset via RAG, generates a structured multi-dish menu using a free LLM API, and automatically evaluates the menu using an LLM-as-judge framework.

T3.1 Knowledge Base Construction (5 marks)

Build a retrieval index over the **full dataset** (train + dev + test) to power your RAG pipeline:

- Convert the datasets into a suitable format for retrieval.
- Encode the dataset into an index for retrieval. You may choose keyword-based (BM25), dense embedding, or a hybrid system.
- Implement retrieval logic to query your index and return top-5 recipes given a user query.
- Report **KB statistics**: number of indexed documents, indexing time, average document length, and retrieval latency for a sample query.

Demonstration Queries:

“Give me some chicken recipes.”
“Give me some dessert recipes.”
“Give me some Italian recipes.”

Evaluation (5 marks — assessed via notebook)

- **2 marks** — Dataset converted into a suitable retrieval format; KB statistics reported (document count, indexing time, latency).
- **2 marks** — Working retrieval index and logic that returns relevant recipes for user queries.
- **1 mark** — Retrieval examples for three test queries with commentary on retrieval quality.

T3.2 RAG Pipeline for Menu Generation

(2 marks)

Implement a pipeline that retrieves relevant recipes and generates a structured multi-dish menu satisfying specified constraints.

Pipeline Components

1. **Retrieval:** Query your knowledge base to retrieve relevant recipes based on the user’s request.
2. **Prompt construction:** Build a prompt that includes retrieved recipes as context and instructs the LLM to generate a menu satisfying the constraints.
3. **Generation:** Call a free-tier LLM API to produce the menu. Recommended options:
 - **Groq**⁵ — fast inference, generous free tier
 - **OpenRouter**⁶ — free-model tier with broad model selection
 - **Nvidia NIM API**⁷ — free-model tier inference

Store your API key as an environment variable; do **not** hardcode it in your notebook.

4. **Output:** A structured menu satisfying the following properties:
 - Courses are clearly labelled (e.g. Starter / Main / Dessert).
 - Each dish is linked to at least one retrieved source recipe by title.

⁵<https://console.groq.com/docs/quickstart>

⁶<https://openrouter.ai/openrouter/free>

⁷<https://build.nvidia.com/>

- Each stated constraint is explicitly acknowledged (e.g. “This dish is low-fat because...”).

Demonstration Queries

Your pipeline must handle all of the following queries. Demonstrate each in your notebook showing the retrieved recipes, generated menu, and judge scores (T3.3).

Level 1 — Simple (no dietary constraints):

“Suggest a 3-course dinner menu: a soup starter, a chicken main dish, and a cake for dessert.”

Level 2 — Intermediate (single constraint).

“Plan a 3-course Italian dinner (starter, pasta main, dessert) for a dinner party.”

Level 3 — Hard (multiple simultaneous constraints):

“Create a low-fat, vegetarian 3-course dinner (soup, main, dessert). Avoid using cheese or cream.”

Evaluation (2 marks — assessed via notebook)

- **2 marks** — Advanced extensions (e.g. query rewriting, multi-turn agentic retrieval, few-shot prompting).

T3.3 LLM-as-Judge Evaluation

(3 marks)

LLM-as-judge is a paradigm in which a large language model is used as an automated evaluator to score the outputs of another (or the same) LLM [5]. Rather than relying solely on human annotators, an LLM judge is prompted with a rubric and asked to assess quality along specified dimensions. This approach is increasingly used in NLP research for scalable, automated evaluation [5].

Design and implement an automated evaluation framework that scores each generated menu using a judge LLM (same or different free API as T3.2). The judge must evaluate menus on **four criteria**, each scored 1–5:

Criterion	Description
Constraint Satisfaction	Does the menu strictly satisfy all stated dietary, cultural, and exclusion constraints?
Ingredient Faithfulness	Do the generated recipe steps use ingredients from retrieved source recipes without hallucinating core ingredients not in the source?
Culinary Logic & Coherence	Are the cooking steps physically plausible and logically ordered (e.g. preheat before baking, correct sequencing, realistic timings)?
Bias	Does the response exhibit unwarranted cultural, dietary, or demographic bias beyond what the user’s request justifies?

For each criterion, the judge must return a score (1–5) and a brief reasoning string. Parse the output into a structured format (e.g. a Python dataclass or dictionary) and display the scores and reasoning for each demonstration menu.

You must run **two rounds of evaluation** for each menu: (1) **self-evaluation** using the same LLM that generated the menu, and (2) **cross-evaluation** using a different LLM. Report both in the table below.

Optional extension (+1 mark bonus, capped at T3.3 total): Conduct a small human evaluation on ≥ 3 menus and report Cohen’s κ or Krippendorff’s α between human and LLM judges to quantify inter-rater reliability.

Menu / Query	Evaluation Type	Judge Model	Constraint	Faithfulness	Logic & Coherence	Bias
Menu 1	Self-evaluation	[Generator LLM Name]				
Menu 1	Cross-evaluation	[External LLM Name]				
Menu 2	Self-evaluation	[Generator LLM Name]				
Menu 2	Cross-evaluation	[External LLM Name]				
Menu 3	Self-evaluation	[Generator LLM Name]				
Menu 3	Cross-evaluation	[External LLM Name]				

Evaluation (3 marks — assessed via notebook)

- **2 marks** — Implementation: both self-evaluation and cross-evaluation run for each menu, outputs parsed into structured scores and displayed.
- **1 mark** — Reported scores are plausible and the judge’s reasoning is consistent with the generated menu content.

T3.4 Analysis and Reflection

(15 marks)

Write a substantive analysis in your report covering the following three areas.

(a) RAG pipeline design (5 marks)

Describe the design of your RAG pipeline in your report. Your description should cover all three stages and LLM as judge design, including the following points:

- **Knowledge base construction** — how you processed the Cooking Dataset for retrieval.
- **Indexing** — how you encoded the dataset for retrieval.
- **RAG** — how your pipeline understands user queries, retrieves relevant recipes, constructs prompts, and generates menus. What specific design choices did you make at each stage, and why?
- **LLM-as-judge** — how you designed the judge prompt and structured the output for scoring.

You may include a **flowchart or architecture diagram** to illustrate the end-to-end pipeline (query → retrieval → generation → judge). This is optional but recommended — a clear diagram can replace a substantial amount of prose.

(b) Per-level evaluation (6 marks)

For each of the three difficulty levels, present: the query used, the top retrieved recipes (titles and a brief summary), the generated menu (full text), and the judge scores with reasoning for each criterion. Then reflect: did the system perform well? Where did it fail? Did the judge’s assessment align with your own judgment?

Also briefly justify your retrieval approach (sparse / dense / hybrid) and when each would be preferable.

A report template with the required per-level structure is provided in `report_template.md`.

(c) Critical reflection on LLM-as-judge (4 marks)

Discuss at least two failure modes of using an LLM to evaluate LLM-generated output (e.g. position bias [5], sycophancy [6], inability to verify factual constraints). How might you improve judge reliability (e.g. stronger LLMs, multi-sampling, rubric calibration, human-in-the-loop spot-checking)? Provide at least one example from your experiments where the judge succeeded and one where it was wrong or inconsistent.

Evaluation (15 marks — assessed via report)

- **5 marks** — RAG pipeline design described clearly across all three stages (KB construction, indexing, RAG, LLM-as-judge prompt); diagram present and accurate (if included).
- **6 marks** — Per-level evaluation presented clearly for all three levels with query, retrieval, generation, and judge scores; includes retrieval approach justification.
- **4 marks** — Critical reflection on LLM-as-judge limitations and improvement strategies, supported by evidence from your experiments.

GPU is not necessary. All retrieval (depending on the method), LLM API calls, and judge scoring run on CPU. Reserve your remaining Colab units for Tasks 1 and 2. If you have remaining compute units, you are welcome to use them for task 3.

Evaluation Metrics (Tasks 1 and 2)

The full evaluation script is provided as `eval_metrics.py` in the assignment materials.

Week-12 Interview

[24 marks]

Interviews are held in Week 12. Each interview is 10–15 minutes and consists of two components.

Implementation Review

(19 marks)

The interviewer will ask targeted questions about your implementations across Tasks 1–3. You should be able to explain and defend your design choices.

Menu Designer Demo

(5 marks)

You must demonstrate your Menu Designer live during the interview. The interviewer will provide a query (which may differ from the queries in your notebook) and observe your system retrieving recipes, generating a menu, and running the LLM-as-judge evaluation in real-time.

Your demo environment must be ready **before** the interview — have your knowledge base pre-built and your API key configured. Cold-starting or re-indexing during the interview will not be accommodated. Your marks will be judged by the level of query difficulty your system can handle and the quality of the generated response.

Evaluation (24 marks total)

- **19 marks** — Implementation review: ability to explain and defend your code across Tasks 1–3 (see table above).
- **5 marks** — Demo: system produces a plausible, constraint-satisfying menu for the interviewer’s query; judge scores are displayed and discussed.

Mark Allocation Summary

Component	Marks
Task 1 — RNN from Scratch (24 marks total)	
T1.1 — Data Preprocessing	2
T1.2 — Baseline RNN Encoder-Decoder	5
T1.3 — RNN with Bahdanau Attention	5
T1.4 — Analysis and Reflection	12
Task 2 — Pre-trained Models + LoRA (27 marks total)	
T2.1 — T5 Fine-tuning with LoRA	6
T2.2 — GPT-2 Fine-tuning with LoRA	6
T2.3 — Analysis and Reflection	15
Task 3 — Menu Designer (25 marks total)	
T3.1 — Knowledge Base Construction	5
T3.2 — RAG Pipeline	2
T3.3 — LLM-as-Judge	3
T3.4 — Analysis and Reflection	15
Interview (24 marks total)	
Implementation Review	19
Menu Designer Demo	5
Total	100

Submission Checklist

File	Task	Notes
Code notebooks / files		
☐ <code>code_studentid.ipynb</code> or <code>A2.T1_studentid.ipynb</code> , <code>A2.T2_studentid.ipynb</code> , <code>A2.T3_studentid.ipynb</code>	All tasks	You may submit either (i) one combined notebook containing all Tasks 1–3, or (ii) separate task-level notebooks / code files for Task 1, Task 2, and Task 3. In either format, outputs must be retained, all submitted notebooks must have all cells executed and visible, and the complete submission must remain reproducible. If a later task depends on artefacts produced earlier (e.g. checkpoints, KB indices, predictions), clearly document how those artefacts are loaded from Google Drive or regenerated. Replace <code>studentid</code> with your actual Monash student ID; if you use non-notebook code files, follow the same naming pattern (e.g. <code>A2.T1_30001234.py</code>).
Report		
☐ <code>report_studentid.pdf</code>	T1.4, T2.3, T3.4	Written analysis. Maximum 12 pages excluding references.
Predictions		
☐ <code>generated_studentid.xlsx</code>	T1.2, T1.3, T2.1, T2.2	Test-set predictions from all four models. See XLSX format below.
Generative AI Declaration (if applicable)		
☐ <code>AI_chat_history_studentid.pdf</code>	All	Required if you used any generative AI tools. Submit the full chat history in PDF following the format specified here .

XLSX format. `generated_studentid.xlsx` must have the following columns, one row per test example in the same order as `test.csv`:

Column	Description
<code>Title</code>	Recipe title from <code>test.csv</code>
<code>Ingredients</code>	Ingredients from <code>test.csv</code>
<code>Recipe_Gold</code>	Gold recipe from <code>test.csv</code>
<code>Recipe_T1_Baseline</code>	Generated recipe — Task 1 Baseline RNN
<code>Recipe_T1_Attention</code>	Generated recipe — Task 1 RNN + Attention
<code>Recipe_T2_T5</code>	Generated recipe — Task 2 T5 + LoRA
<code>Recipe_T2_GPT2</code>	Generated recipe — Task 2 GPT-2 + LoRA

Before submitting, confirm all of the following:

- ☐ All filenames use your actual student ID in place of `studentid`.
- ☐ If you submit a single combined notebook, `code_studentid.ipynb` has a Markdown cell at the top with your name, student ID, and a table of contents. If you submit separate task-level notebooks / files, each submitted task file must clearly state your name, student ID, and the task it covers; notebook submissions should also include a short contents overview.
- ☐ All submitted notebook cells are executed and outputs are visible. Training logs, loss curves, and metric results are present where relevant. Do **not** clear outputs before submission.
- ☐ Observations and justifications are written in Markdown cells immediately following the relevant outputs, not in code comments.
- ☐ Random seeds are set (`torch.manual_seed(42)`) and all key hyperparameters are documented.
- ☐ Code is readable and modular; comments added only where logic is non-obvious.
- ☐ If you submit separate task-level files, the dependency between files is explicit: note any required prerequisite artefacts, their Google Drive paths / filenames, and the steps needed to reload or regenerate them.
- ☐ Your API key is stored as an environment variable and is **not** hardcoded in the notebook.
- ☐ Your Menu Designer demo environment is ready: knowledge base pre-built, API key configured.
- ☐ If you used any methods not taught in class, you have provided citations and justification.
- ☐ Your demo environment is ready **before** the Week-12 interview; do not rely on re-indexing during the interview.

Academic Integrity and Use of Generative AI

This is an individual assessment. You must write your own code and analysis. Discussion with peers about conceptual questions is permitted; sharing code is not.

At the beginning of your jupyter notebook, include a declaration cell indicating your use of generative AI. Select one of the following statements:

1. *I did not use any generative AI tools in any part of this submission.*
2. *I used generative AI tools only for language editing and code formatting. All content and code are my own work.*
3. *I used generative AI tools to assist with coding, understanding the assignment, and supporting the analysis.*

If you select option 2 or 3, you must also provide a summary specifying which AI tools you have used and for what purposes, along with your complete chat history, following the format [here](#). The full chat history should be included in a separate PDF document. Links to external chat logs will not be accepted.

References

- [1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*. [arXiv:1409.0473](#)

- [2] Kiddon, C., Zettlemoyer, L., & Choi, Y. (2016). Globally Coherent Text Generation with Neural Checklist Models. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 329–339.
- [3] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140), 1-67.
- [4] Radford, Alec, et al (2019). "Language models are unsupervised multitask learners." *OpenAI blog*.
- [5] Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... & Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36, 46595-46623.
- [6] Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Aspell, A., Bowman, S. R., ... & Perez, E. (2024) "Towards understanding sycophancy in language models." *The Twelfth International Conference on Learning Representations* (2024).
- [7] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). Lora: Low-rank adaptation of large language models. *The Twelfth International Conference on Learning Representations* (2022).
- [8] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33, 9459-9474.